

BookKaroyar - Master Product & Engineering Architecture

Version 1.0 - implementation source of truth

SOURCE OF TRUTH: This resource intentionally uses plain JavaScript. The locked stack is React + JavaScript + Firebase + Material UI (MUI). Do not introduce Next.js, TypeScript, shadcn/ui, Redux, or another backend framework unless the project owner explicitly changes this specification.

1. Product statement

BookKaroyar is a collaborative planning application for friends and corporate groups. A group can create a plan such as a movie night, dinner, event, or trip; invite members through a shareable link; coordinate activities and bookings; track shared expenses; and settle balances.

2. Primary user journey

Sign up -> Create Group -> Invite Members -> Create Plan -> Vote / Coordinate -> Add Booking -> Add Expense -> Auto-calculate Splits -> Settle -> Archive / Keep History

3. Locked technology stack

Layer	Technology	Rule
Frontend	React.js + JavaScript	No TypeScript
Routing	React Router	Protected routes required
UI	Material UI (MUI)	Primary component library
Styling	Tailwind CSS optional	Use for layout/utility; do not create competing design systems
Auth	Firebase Authentication	Email/password + Google
Database	Cloud Firestore	Persistent app data
Files	Firebase Storage	Avatars / plan media
Backend logic	Firebase Cloud Functions	Only for trusted server-side logic
Realtime	Firestore listeners	Use for collaborative screens
Hosting	Firebase Hosting or equivalent	SPA deployment

4. Domain model

User
has GroupMemberships
has direct Notifications

Group
has Members
has Plans
has Expenses
has Bookings
has Polls
has Activities
has InviteLinks

Plan
belongs to Group
has ItineraryItems
has Bookings
has Expenses

5. Global engineering rules

- UI components must not contain raw Firestore queries. Put Firebase operations in feature services/hooks.
- Every create/update/delete action must define loading, success, error, and empty behavior where applicable.
- Every group-scoped read/write must be protected by Firestore Security Rules, not only by client-side checks.
- Never trust client-calculated permissions, balances, or admin status for privileged operations.
- Use server timestamps for auditable events.
- Use real IDs in relationships; do not rely on display names or email strings as primary keys.
- Destructive actions require confirmation unless explicitly marked as reversible.
- Demo seed data must never be mixed with real user data.

6. Suggested project structure

```
src/  
components/  
layout/  
common/  
features/  
auth/  
dashboard/  
groups/  
invites/  
plans/  
bookings/  
expenses/  
settlements/  
polls/  
notifications/  
hooks/  
services/  
firebase/  
auth/  
groups/  
plans/  
bookings/  
expenses/  
routes/  
utils/  
App.jsx  
main.jsx
```

7. Definition of done

- Feature works with seeded data and with a new account.
- Feature survives page refresh because data is persistent.
- Feature is usable on desktop and mobile widths.
- Feature has loading, empty, error, and success feedback.
- Unauthorized users cannot access group-scoped data by manipulating the client.
- No console errors in normal workflows.
- No fake placeholder buttons that appear functional but do nothing.